

Create by: Shivakant Kurmi

Exercise 1: Setting Up JUnit in Java using Eclipse

Calculator.java

//Name: Shivakant Kurmi (shivkant.23bcg10140@vitbhopal.ac.in)

package javv;

public class Calculator {

public static int calculate(**int** a , **int** b) {

return a*b;

}

public String concat(String a ,String b) {

return a + b;

}

}

Concat.java

package javv;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

public class concat {

 @Test

public void concatTest() {

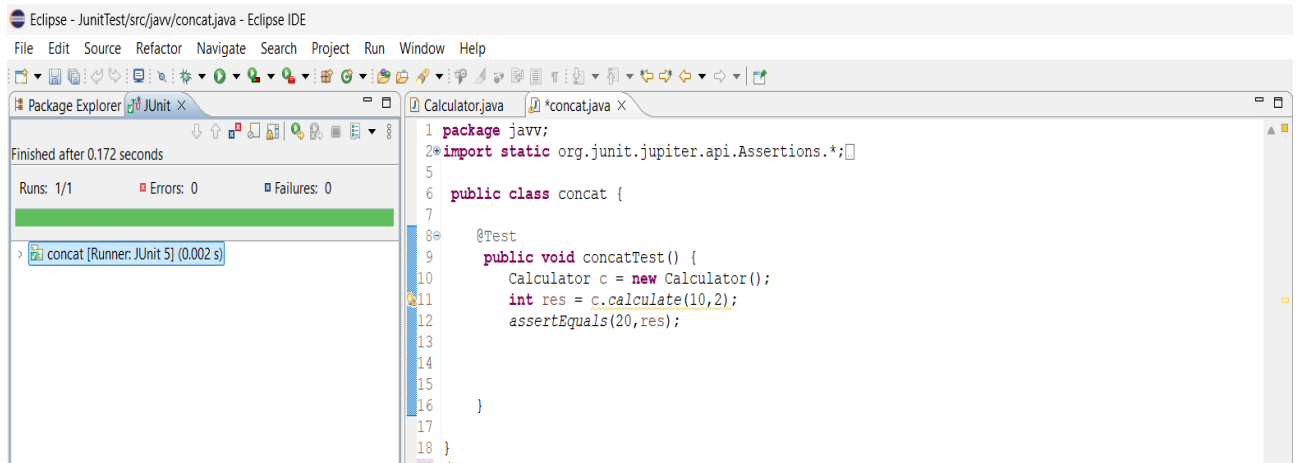
 Calculator c = new Calculator();

int res = c.calculate(10,2);

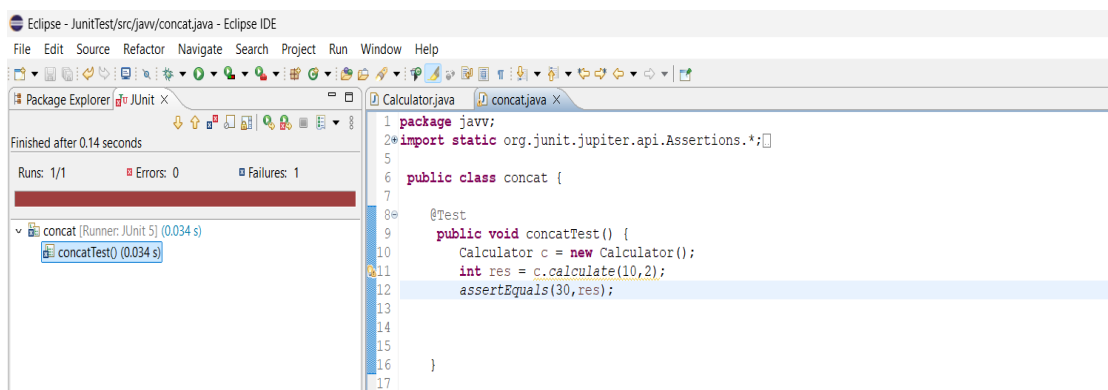
 assertEquals(20,res);

 }

}

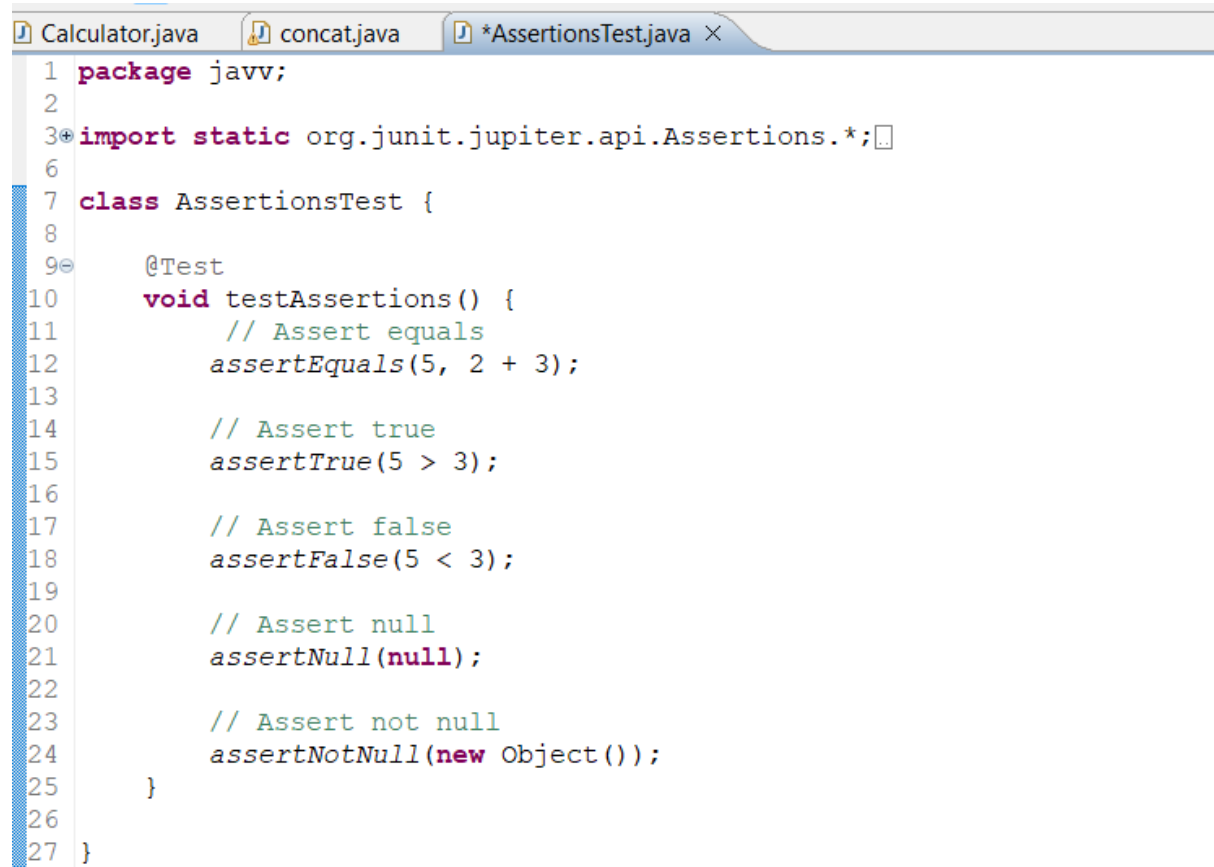


Error in Junit testing

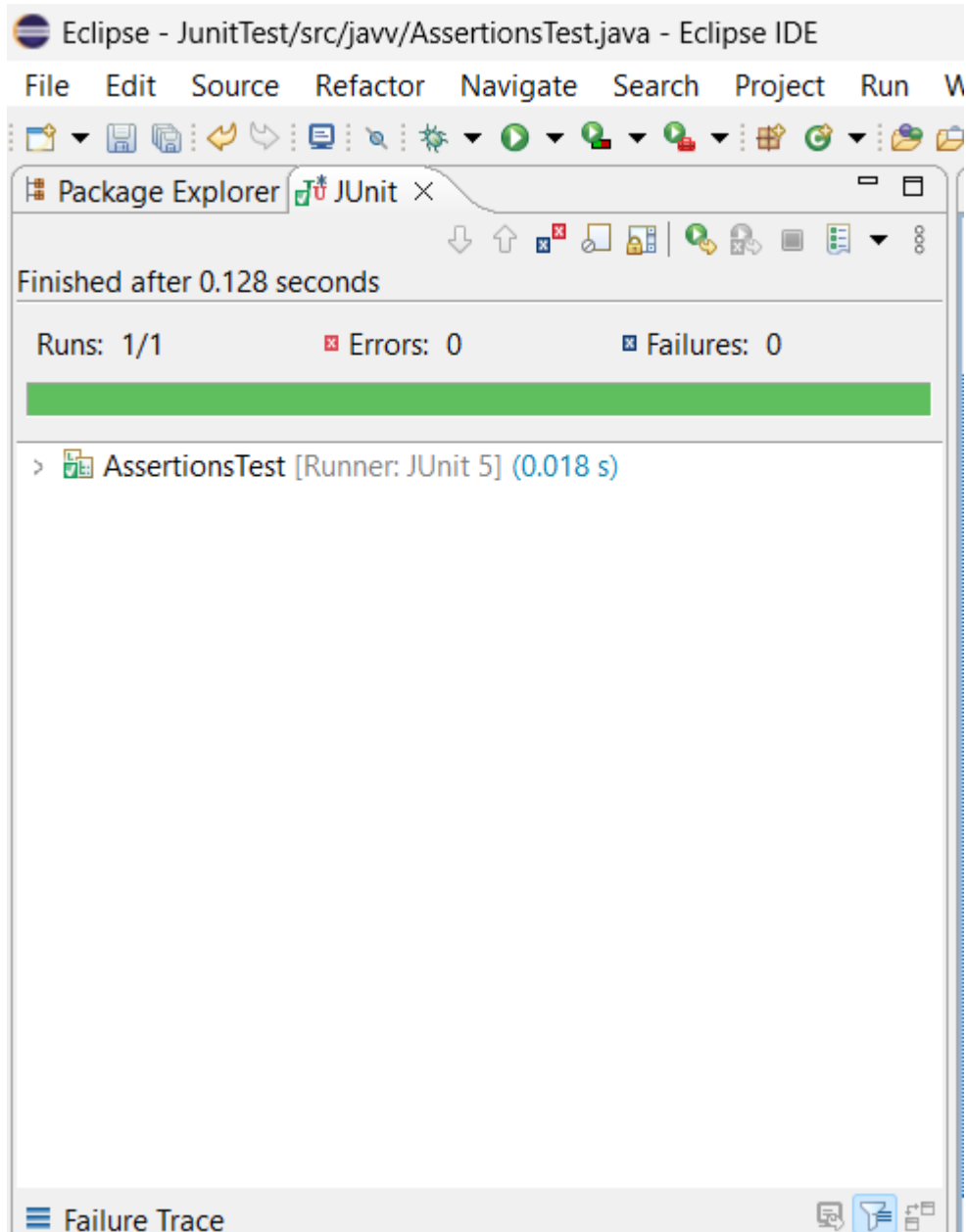


Exercise 3: Assertions in Junit

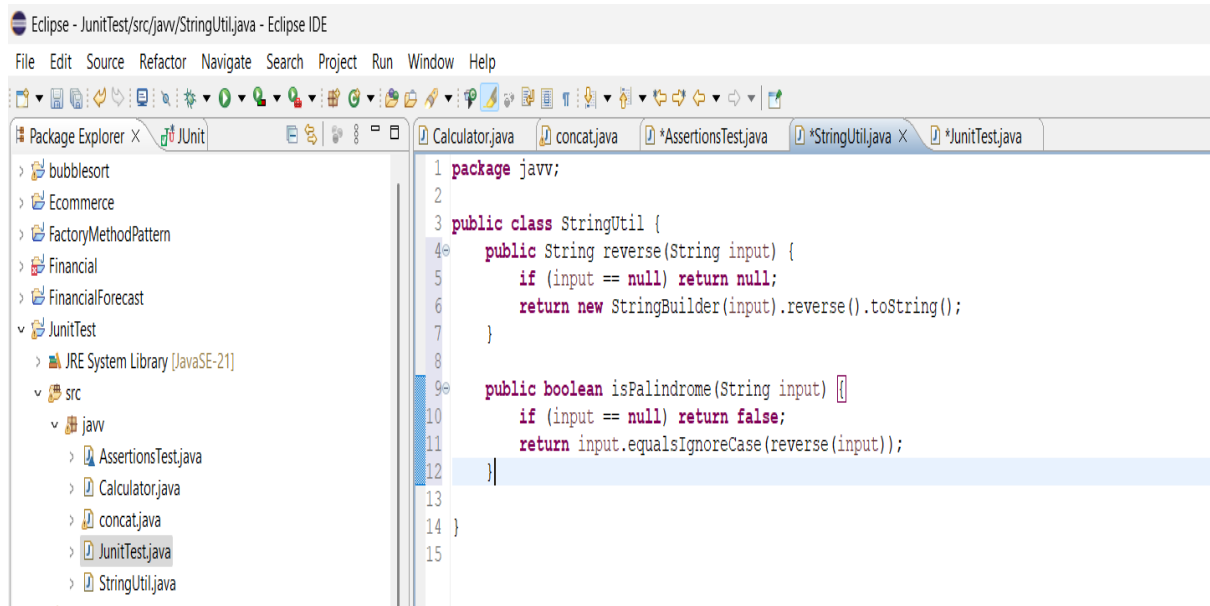
AssertionsTest.java



```
1 package javv;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5
6
7 class AssertionsTest {
8
9     @Test
10     void testAssertions() {
11         // Assert equals
12         assertEquals(5, 2 + 3);
13
14         // Assert true
15         assertTrue(5 > 3);
16
17         // Assert false
18         assertFalse(5 < 3);
19
20         // Assert null
21         assertNull(null);
22
23         // Assert not null
24         assertNotNull(new Object());
25     }
26
27 }
```

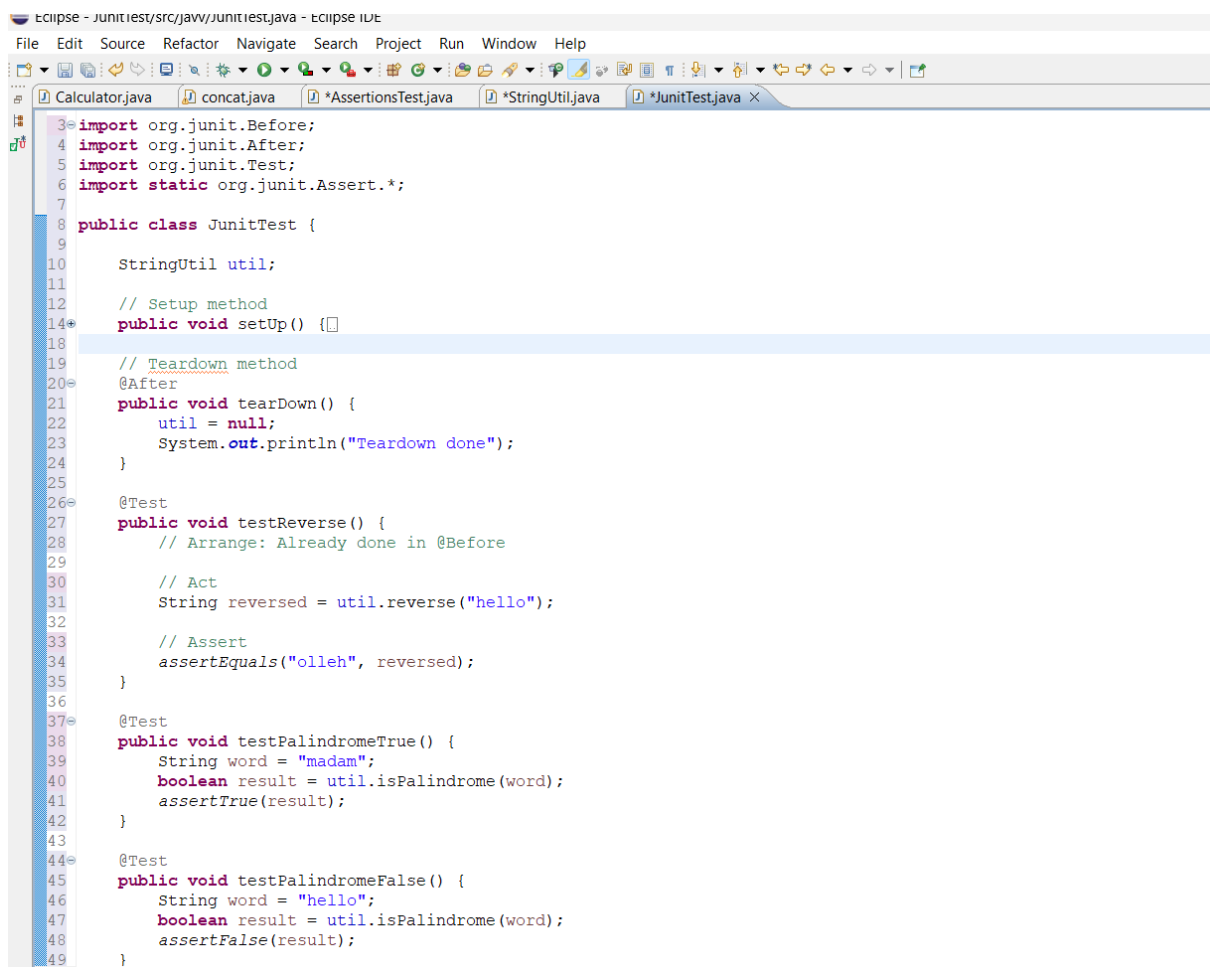


Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown in JUnit



The screenshot shows the Eclipse IDE with the file `StringUtil.java` open. The Package Explorer on the left shows the project structure: `bubblesort`, `Ecommerce`, `FactoryMethodPattern`, `Financial`, `FinancialForecast`, `JUnitTest` (selected), `JRE System Library [JavaSE-21]`, `src`, `javv`, `AssertionsTest.java`, `Calculator.java`, `concat.java`, `JUnitTest.java`, and `StringUtil.java`. The main editor displays the following code:

```
1 package javv;
2
3 public class StringUtil {
4     public String reverse(String input) {
5         if (input == null) return null;
6         return new StringBuilder(input).reverse().toString();
7     }
8
9     public boolean isPalindrome(String input) {
10        if (input == null) return false;
11        return input.equalsIgnoreCase(reverse(input));
12    }
13
14 }
15
```



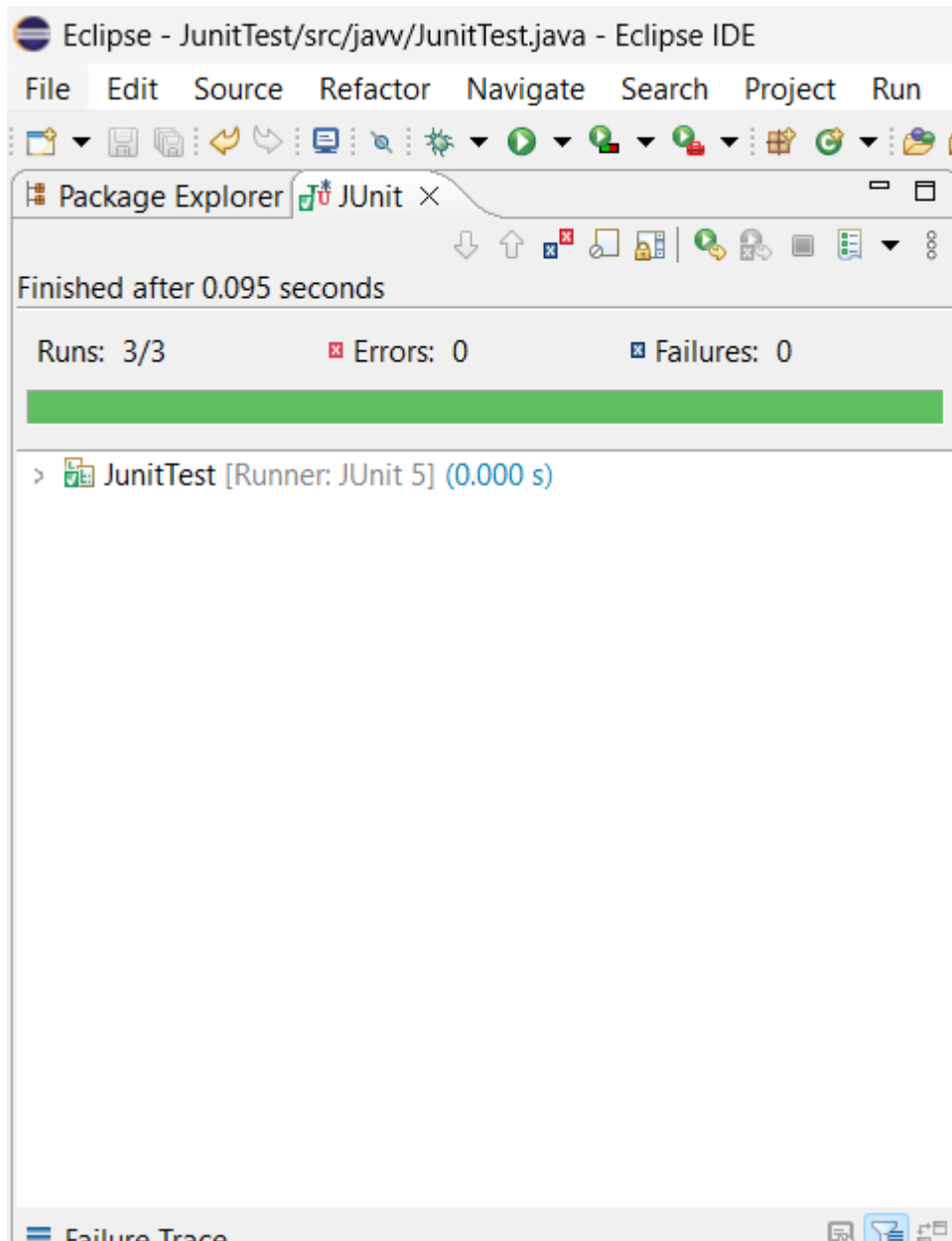
The screenshot shows the Eclipse IDE with the file `JUnitTest.java` open. The Package Explorer on the left shows the project structure: `bubblesort`, `Ecommerce`, `FactoryMethodPattern`, `Financial`, `FinancialForecast`, `JUnitTest` (selected), `JRE System Library [JavaSE-21]`, `src`, `javv`, `AssertionsTest.java`, `Calculator.java`, `concat.java`, `JUnitTest.java`, and `StringUtil.java`. The main editor displays the following code:

```
3 import org.junit.Before;
4 import org.junit.After;
5 import org.junit.Test;
6 import static org.junit.Assert.*;
7
8 public class JUnitTest {
9
10    StringUtil util;
11
12    // Setup method
13    public void setUp() {
14
15    }
16
17    // Teardown method
18    @After
19    public void tearDown() {
20        util = null;
21        System.out.println("Teardown done");
22    }
23
24    @Test
25    public void testReverse() {
26        // Arrange: Already done in @Before
27
28        // Act
29        String reversed = util.reverse("hello");
30
31        // Assert
32        assertEquals("olleh", reversed);
33    }
34
35    @Test
36    public void testPalindromeTrue() {
37        String word = "madam";
38        boolean result = util.isPalindrome(word);
39        assertTrue(result);
40    }
41
42    @Test
43    public void testPalindromeFalse() {
44        String word = "hello";
45        boolean result = util.isPalindrome(word);
46        assertFalse(result);
47    }
48
49 }
```

Problems | Javadoc | Declaration | Console x

<terminated> JUnitTest [JUnit] C:\Program Files\Java\jdk-21\bin\javaw.exe (Jun 28, 2025, 10:48:01 PM – 10:48:04 PM) [pid: 13148]

Setup done
Teardown done
Setup done
Teardown done
Setup done
Teardown done



Error :

Eclipse - JUnitTest/src/javv/JunitTest.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer JUnit ×

inished after 0.101 seconds

Runs: 3/3 Errors: 0 Failures: 1

JUnitTest [Runner: JUnit 5] (0.001 s)

- testPalindromeTrue (0.001 s)
- testReverse (0.000 s)
- testPalindromeFalse (0.000 s)

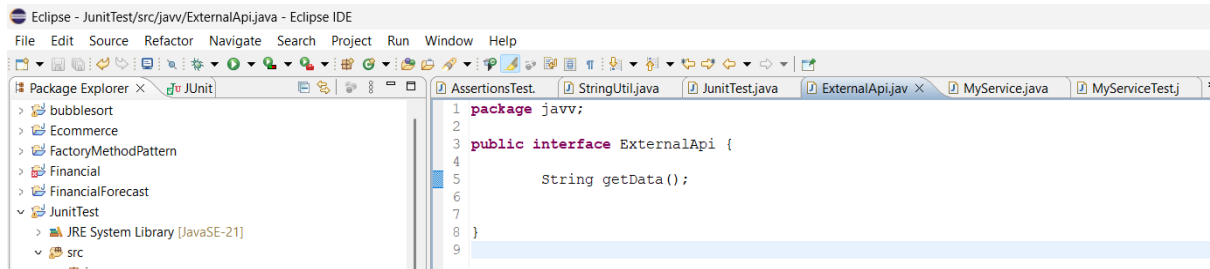
Failure Trace

- java.lang.AssertionError
- at org.junit.Assert.fail(Assert.java:87)

```
21 public void tearDown() {
22     util = null;
23     System.out.println("Teardown done");
24 }
25
26 @Test
27 public void testReverse() {
28     // Arrange: Already done in @Before
29
30     // Act
31     String reversed = util.reverse("hello");
32
33     // Assert
34     assertEquals("olleh", reversed);
35 }
36
37 @Test
38 public void testPalindromeTrue() {
39     String word = "mada";
40     boolean result = util.isPalindrome(word);
41     assertTrue(result);
42 }
43
44 @Test
45 public void testPalindromeFalse() {
46     String word = "hello";
47     boolean result = util.isPalindrome(word);
48     assertFalse(result);
49 }
```

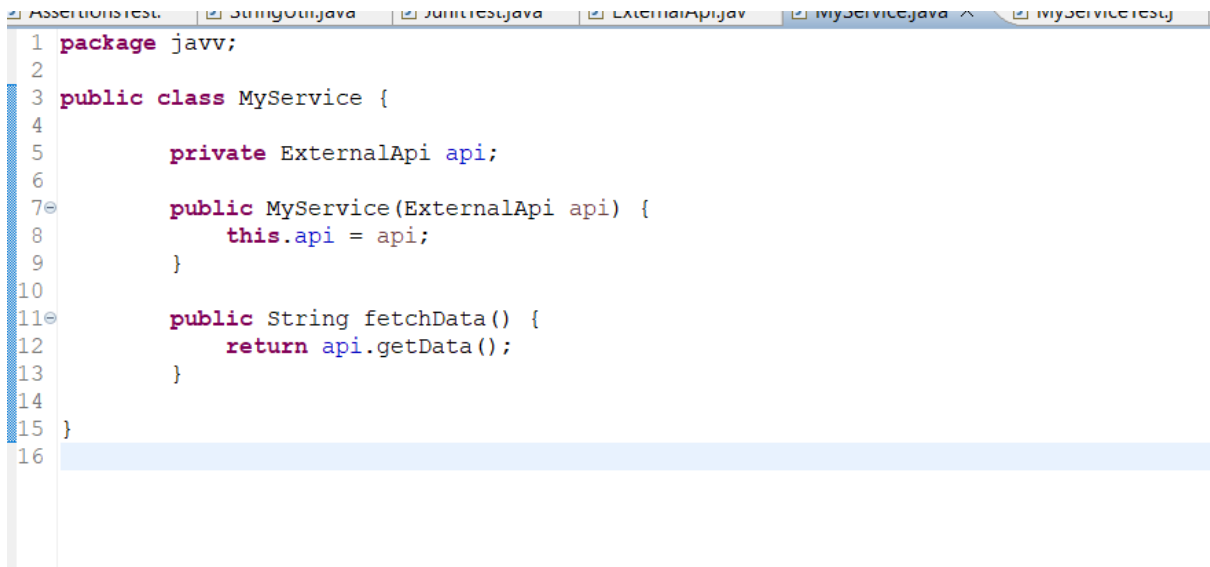
. Mockito exercises

Exercise 1: Mocking and Stubbing



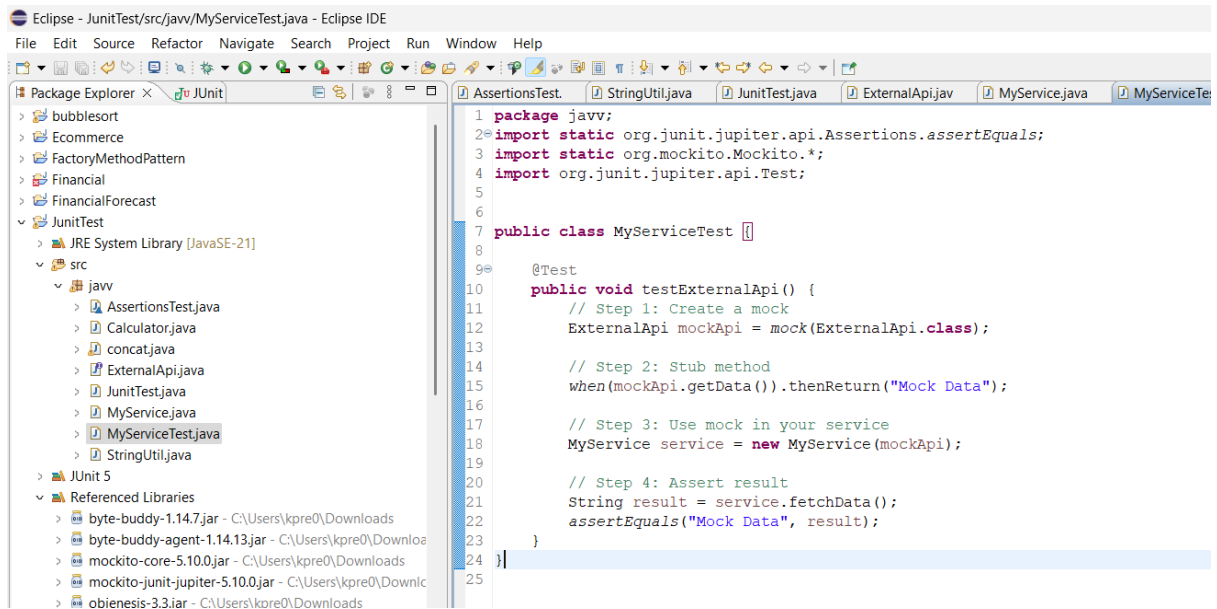
This screenshot shows the Eclipse IDE with the file `ExternalApi.java` open. The Package Explorer on the left shows the project structure, including the `JUnit` package. The main editor displays the following code:

```
1 package javv;
2
3 public interface ExternalApi {
4
5     String getData();
6
7 }
8
9
```



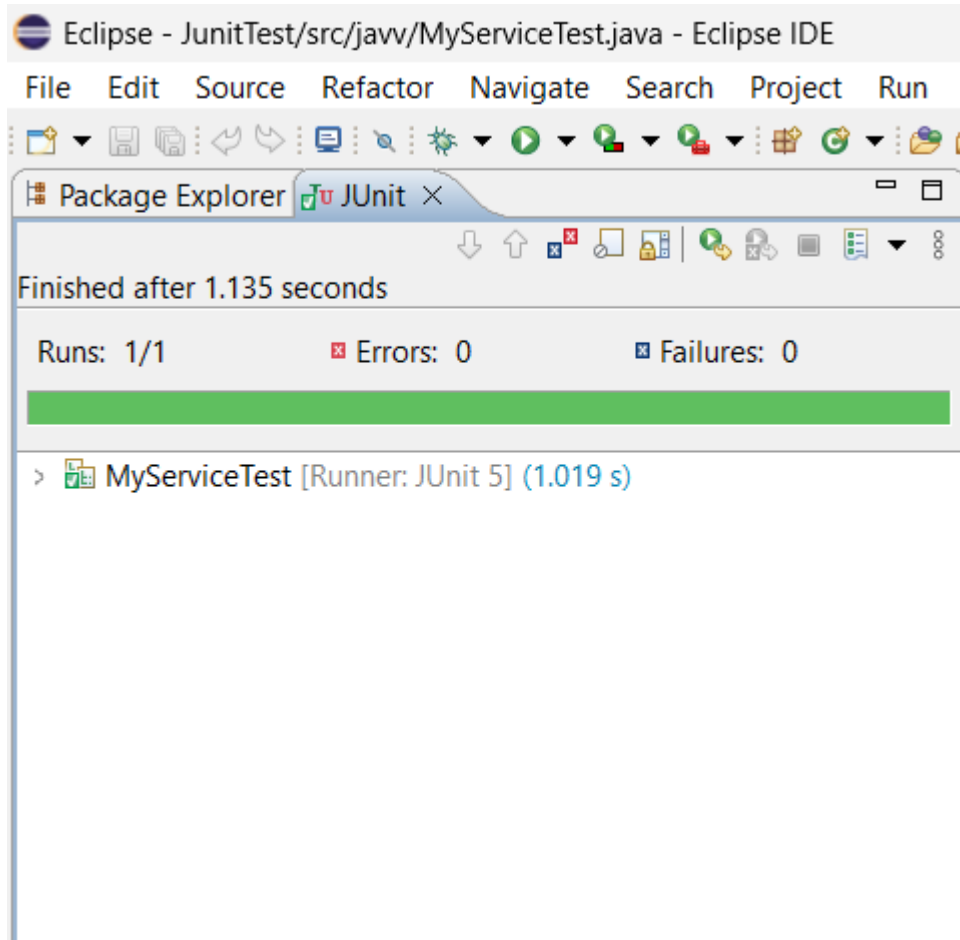
This screenshot shows the Eclipse IDE with the file `MyService.java` open. The main editor displays the following code:

```
1 package javv;
2
3 public class MyService {
4
5     private ExternalApi api;
6
7     public MyService(ExternalApi api) {
8         this.api = api;
9     }
10
11     public String fetchData() {
12         return api.getData();
13     }
14
15 }
16
```



This screenshot shows the Eclipse IDE with the file `MyServiceTest.java` open. The Package Explorer on the left shows the project structure, including the `JUnit` package and the `src` package. The main editor displays the following code:

```
1 package javv;
2 import static org.junit.jupiter.api.Assertions.assertEquals;
3 import static org.mockito.Mockito.*;
4 import org.junit.jupiter.api.Test;
5
6
7 public class MyServiceTest {
8
9     @Test
10     public void testExternalApi() {
11         // Step 1: Create a mock
12         ExternalApi mockApi = mock(ExternalApi.class);
13
14         // Step 2: Stub method
15         when(mockApi.getData()).thenReturn("Mock Data");
16
17         // Step 3: Use mock in your service
18         MyService service = new MyService(mockApi);
19
20         // Step 4: Assert result
21         String result = service.fetchData();
22         assertEquals("Mock Data", result);
23     }
24 }
25
```

Exercise 2: Verifying Interactions

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>mockitotest</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <dependencies>

    <!-- JUnit 5 -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.0</version>
      <scope>test</scope>
    </dependency>

    <!-- Mockito -->
    <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>5.7.0</version>
      <scope>test</scope>
    </dependency>
```

```
</dependencies>
```

```
<build>
```

```
<plugins>
```

```
<!-- Enable JUnit 5 with Maven -->
```

```
<plugin>
```

```
<groupId>org.apache.maven.plugins</groupId>
```

```
<artifactId>maven-surefire-plugin</artifactId>
```

```
<version>3.0.0-M9</version>
```

```
</plugin>
```

```
</plugins>
```

```
</build>
```

```
</project>
```

MockitoTest.java

```
package com.example;
```

```
import org.junit.jupiter.api.Test;
```

```
import static org.mockito.Mockito.*;
```

```
class ExternalApi {
```

```
    void getData() {
```

```
        // mock method
```

```
    }
```

```
}
```

```
class MyService {
```

```
    private ExternalApi api;
```

```

// Constructor that accepts a mock or real object
public MyService(ExternalApi api) {
    this.api = api;
}

public void fetchData() {
    api.getData(); // This will throw NullPointerException if api is null
}
}

public class mockitoTest {

    @Test
    void testVerifyInteraction() {
        // Step 1: Create mock
        ExternalApi mockApi = mock(ExternalApi.class);

        // Step 2: Inject the mock into MyService
        MyService service = new MyService(mockApi);

        // Step 3: Call the method that should use the mock
        service.fetchData();

        // Step 4: Verify that getData() was called on the mock
        verify(mockApi).getData();
    }
}

```

